New Horizons®
Centro de Capacitación

Curso

Desarrollo Full Stack con C# y Angular

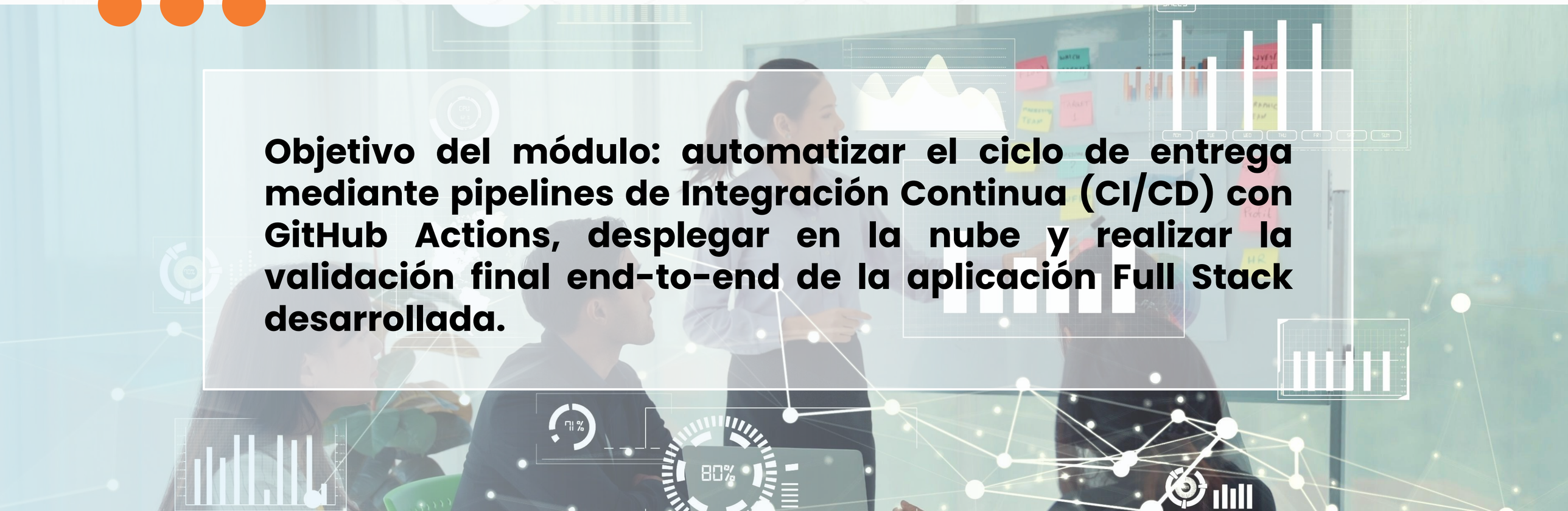
Docente: Edwin Barrientos



New Horizons®
Centro de Capacitación

Sesión 19 — Módulo 5:

CI/CD con GitHub Actions, Despliegue Cloud y Cierre



Objetivo del módulo: automatizar el ciclo de entrega mediante pipelines de Integración Continua (CI/CD) con GitHub Actions, desplegar en la nube y realizar la validación final end-to-end de la aplicación Full Stack desarrollada.



New Horizons®
Centro de Capacitación

Agenda

- Fundamentos de DevOps y entrega continua: Pipelines de CI/CD automatizados
- Configuración de GitHub Actions con .github/workflows/ci-cd.yml para .NET y Angular
- Automatización de pruebas unitarias (dotnet test) y verificación de compilación (ng build)
- Estrategias de despliegue en la nube: Azure App Service, AWS, Render y VPS
- Prueba de regresión integral end-to-end del flujo con los 4 roles y retrospectiva final



New Horizons®
Centro de Capacitación

Sesión 1:

CI/CD con GitHub Actions, Despliegue Cloud y Presentación Final

Visión general de la sesión

New Horizons
Centro de Capacitación

Hoy convertimos tu proyecto en una línea de ensamblaje. Cada commit entra por un extremo y sale por el otro probado, empaquetado y desplegado en la nube, sin que nadie tenga que “armarlo a mano”.

Integración Continua

Cada commit se compila y se prueba solo, en un servidor limpio.

Piénsalo así: *El control de calidad de la fábrica: ninguna pieza defectuosa avanza en la línea.*

Entrega Continua

Publicar una nueva versión toma minutos, no una tarde entera.

Piénsalo así: *El camión de reparto sale solo cuando la guía está firmada.*

Validación Final

Regresión end-to-end del flujo completo con los 4 roles.

Piénsalo así: *El ensayo general antes del estreno: todo el elenco, con luces reales.*

Pilares técnicos fundamentales

New Horizons
Centro de Capacitación

Cinco estaciones de la línea de ensamblaje: el código pasa por cada una antes de llegar al usuario.

1

DevOps y CI/CD

La cinta transportadora

Pipelines automatizados que unen desarrollo y operación.

2

GitHub Actions

El manual de la fábrica

ci-cd.yml define cuándo y cómo corre cada paso para .NET y Angular.

3

Pruebas y build

Control de calidad

dotnet test y ng build validan cada cambio en limpio.

4

Despliegue cloud

El camión de reparto

AWS, Azure App Service, Render o VPS: dónde vivirá la app.

5

Regresión E2E

El ensayo general

El flujo completo con los 4 roles y la retrospectiva final.

Si una estación falla, **la línea se detiene**: nada roto llega a producción. Ese es todo el valor del pipeline.

Anatomía de un workflow de GitHub Actions

New Horizons®
Centro de Capacitación

`.github/workflows/ci-cd.yml`

```yaml
name: CI-CD Solicitudes

1 on:
    push:
      branches: [ main ]
    pull_request:
      branches: [ main ]

2 jobs:
    backend:
3     runs-on: ubuntu-latest
4     steps:
        - uses: actions/checkout@v4
        - name: Compilar y probar
          run: dotnet test --configuration Release
```

El guion de una obra

El YAML es el guion: dice cuándo sube el telón, qué actos hay y qué pasa en cada escena.

1 on = cuándo sube el telón

Los eventos que disparan el workflow: push, pull request…

2 jobs = los actos

Bloques independientes; por defecto corren en paralelo.

3 runs-on = el escenario

La máquina virtual limpia donde corre el job.

4 steps = las escenas

Comandos en orden: si uno falla, el acto termina.

Práctica: pipeline de CI para el backend .NET

New Horizons
Centro de Capacitación

`.github/workflows/ci.yml`

```yaml
jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: "8.0.x"
      - name: Restaurar dependencias
        run: dotnet restore
      - name: Compilar en limpio
        run: dotnet build --no-restore -c Release
      - name: Pruebas unitarias
        run: dotnet test --no-build -c Release
```

Estaciones de control de calidad

Cada estación revisa una sola cosa. Si una falla, la cinta se detiene.

1. **checkout**
   Trae el código del commit.
2. **setup-dotnet**
   Instala el SDK exacto.
3. **restore**
   Descarga los paquetes NuGet.
4. **build -c Release**
   Compila como en producción.
5. **test --no-build**
   Prueba lo ya compilado.

CI, Continuous Delivery y Continuous Deployment

New Horizons®
Centro de Capacitación

🥖 **Piensa en una panadería.** Las tres prácticas son la misma cinta; cambia hasta dónde llega el pan sin que una persona intervenga.

CI 🧪

Integración Continua

Se prueba la masa de cada tanda.

- ✓ Cada commit compila y pasa pruebas
- ✓ Feedback en minutos al desarrollador
- ✓ Evita el “en mi máquina funciona”

CD 🧺

Continuous Delivery

El pan queda listo en la vitrina; alguien decide venderlo.

- ✓ Siempre hay una versión desplegable
- ✓ El paso a producción es un clic
- ✓ Aprobación humana antes de salir

CD 🚚

Continuous Deployment

El pan sale directo al cliente, sin esperar.

- ✓ Cada cambio en main va a producción
- ✓ Exige pruebas muy confiables
- ✓ Cero intervención manual

🎯 En el proyecto del curso usamos **Continuous Delivery**: todo automático hasta producción, con un clic de aprobación al final.

Disparadores y protección de la rama main

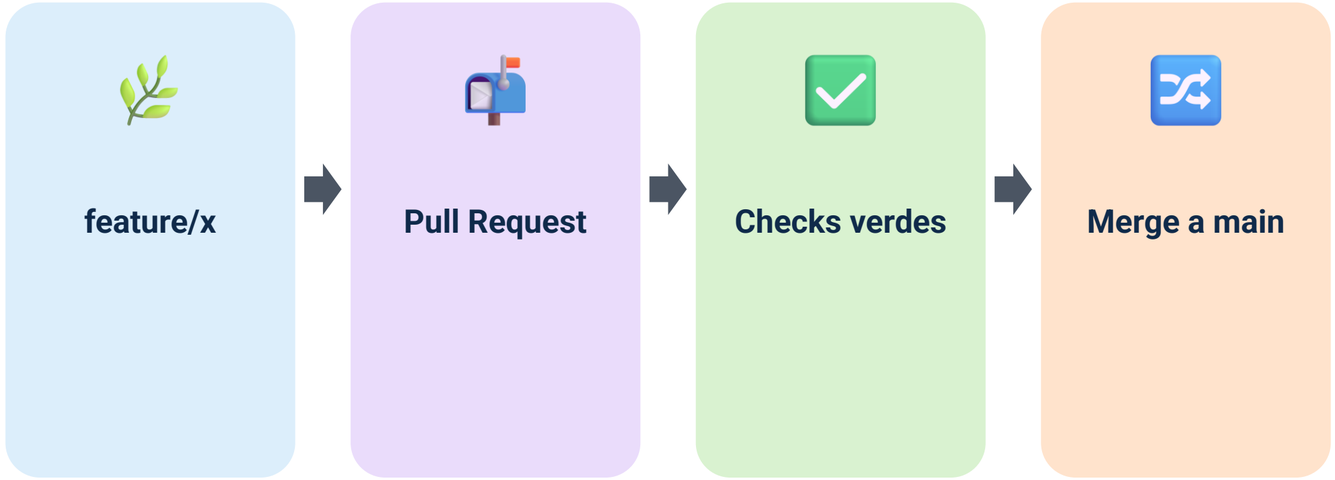
New Horizons
Centro de Capacitación

```
ci-cd.yml  ·  on:

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:   # botón "Run workflow"
```

1
2
3

El camino de un cambio hasta main



El torniquete del metro

Sin pasaje válido (checks en verde) no se entra a main. Da igual quién empuje.

1 **push**

Corre en cada commit a main o develop.

2 **pull_request**

Valida el cambio antes del merge.

3 **workflow_dispatch**

Ejecución manual desde la pestaña Actions.

Settings → Branches → **Require status checks to pass**: así GitHub bloquea el botón Merge si el pipeline falla.

Job de frontend: pruebas y build de Angular

New Horizons
Centro de Capacitación

`.github/workflows/ci-cd.yml`

```yaml
  frontend:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: ./frontend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
          cache-dependency-path: frontend/package-lock.json
      - run: npm ci
      - run: npx ng test --watch=false --browsers=ChromeHeadless
      - run: npx ng build --configuration production
```



El horno de prueba

Horneas una vez en condiciones reales. Si la masa no sube, ese pan no se vende.

1. **working-directory**

   El frontend vive en su propia carpeta.

2. **cache: npm**

   Reutiliza descargas: builds más rápidos.

3. **npm ci**

   Instala exacto lo del lock, sin sorpresas.

4. **ChromeHeadless**

   Navegador sin pantalla para las pruebas.

5. **--configuration production**

   El mismo build que irá a producción.

Matrix build: una receta, varias cocinas

New Horizons
Centro de Capacitación

`.github/workflows/ci.yml`

```yaml
backend:
  strategy:
    fail-fast: false
    matrix:
      os: [ ubuntu-latest, windows-latest ]
      dotnet: [ "8.0.x", "9.0.x" ]
  runs-on: ${{ matrix.os }}
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-dotnet@v4
      with:
        dotnet-version: ${{ matrix.dotnet }}
    - run: dotnet test -c Release
```



Probar la receta en varias cocinas

Gas, inducción, horno grande o chico: si sale bien en todas, la receta es confiable.

2 SO × 2 versiones = 4 jobs en paralelo

✅ ubuntu .NET 8.0	✅ ubuntu .NET 9.0
✅ windows .NET 8.0	❌ windows .NET 9.0

1. **fail-fast: false**
2. **matrix**
3. **${{ matrix.os }}**

Pruebas unitarias con xUnit: dotnet test

New Horizons
Centro de Capacitación

SolicitudTests.cs

```
public class SolicitudTests
{
    [Fact]
    public void Aprobar_DesdePendiente_CambiaAEstadoAprobada()
    {
1       // Arrange
        var solicitud = new Solicitud { Estado = Estado.Pendiente };

2       // Act
        solicitud.Aprobar(aprobadorId: 3, comentario: "Conforme");

3       // Assert
        Assert.Equal(Estado.Aprobada, solicitud.Estado);
    }
}
```

```
$ dotnet test -c Release
Passed!  - Failed: 0, Passed: 24, Skipped: 0, Total: 24
```



Crash test dummies

Chocamos el auto en el laboratorio para no chocarlo con pasajeros reales.

1

Arrange = preparar el escenario

Una solicitud en estado Pendiente.

2

Act = ejecutar la acción

Un aprobador la aprueba con comentario.

3

Assert = verificar el resultado

El estado debe ser Aprobada. Si no, la línea se detiene.



Nombre de la prueba = **Método_Escenario_Resultado**: se lee como una especificación.

Secretos y variables: GitHub Secrets

New Horizons
Centro de Capacitación

`.github/workflows/ci-cd.yml`

```yaml
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    env:
      API_URL: ${{ vars.API_URL }}
    steps:
      - name: Migrar base de datos
        run: dotnet ef database update --project src/Api
        env:
          ConnectionStrings__Default: ${{ secrets.DB_CONNECTION }}
```

1
2
3

Lo que se ve en el log del job

```
ConnectionStrings__Default: ***
Done. Applied 2 migrations.
```



La caja fuerte del hotel

Sabes que la llave existe y la usas, pero nunca queda a la vista: en el log aparece ***.

1 **environment: production**

Reglas propias: aprobadores y secretos solo de producción.

2 **vars vs secrets**

vars para datos públicos (URL); secrets para lo confidencial.

3 **Doble guion bajo __**

Equivale a la jerarquía de appsettings.json en .NET.



Nunca en el repo: ni en el YAML, ni en appsettings, ni en un commit “temporal”.

Artefactos y dependencias entre jobs

New Horizons®
Centro de Capacitación

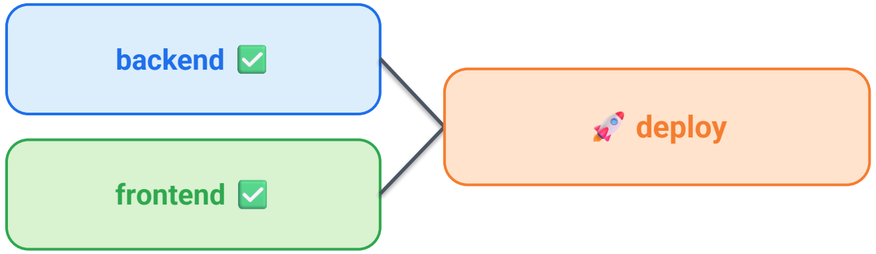
`.github/workflows/ci-cd.yml`

```yaml
frontend:
  steps:
    # ... npm ci, ng test, ng build ...
    - uses: actions/upload-artifact@v4 (1)
      with:
        name: frontend-dist
        path: frontend/dist/

deploy:
  needs: [ backend, frontend ] (2)
  if: github.ref == 'refs/heads/main' (3)
  steps:
    - uses: actions/download-artifact@v4 (4)
      with:
        name: frontend-dist
```

Una carrera de relevos

El testigo (artefacto) pasa de mano en mano. Nadie arranca si el anterior no llegó.



1. **`upload-artifact`** Guarda el build para otros jobs.
2. **`needs`** Espera a que ambos terminen en verde.
3. **`if: main`** Solo se despliega desde la rama principal.
4. **`download-artifact`** Despliega exactamente lo probado.

¿Dónde vivirá tu aplicación? Estrategias de despliegue

New Horizons®
Centro de Capacitación

🏠 **Elegir dónde desplegar es como elegir dónde vivir:** cuanto más administrado, menos te preocupas por la tubería… y menos control tienes.

Opción	Tipo	Es como…	Ideal para	Tú gestionas
☁️ **AWS App Runner / Beanstalk**	PaaS	*Hotel todo incluido*	Escalar sin administrar servidores	Código y configuración
☁️ **Azure App Service**	PaaS	*Hotel todo incluido*	Ecosistemas centrados en .NET	Código y configuración
🟣 **Render**	PaaS simple	*Departamento amoblado*	Demos, MVP y el proyecto del curso	Casi nada
🖥️ **VPS (EC2, Droplet)**	IaaS	*Casa propia*	Control total y costo predecible	SO, parches, Nginx, SSL, backups

Job de despliegue: el frontend a la nube

New Horizons
Centro de Capacitación

`.github/workflows/ci-cd.yml`

```yaml
deploy:
  needs: [ backend, frontend ]
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  environment: production
  permissions:
    id-token: write   # OIDC: sin llaves de larga duración
    contents: read
  steps:
    - uses: actions/download-artifact@v4
      with: { name: frontend-dist, path: dist }
    - uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: ${{ secrets.AWS_ROLE_ARN }}
        aws-region: us-east-1
    - run: aws s3 sync dist/ s3://${{ secrets.S3_BUCKET }} --delete
```



El camión de reparto

Solo sale si control de calidad firmó la guía y la ruta es la oficial (main).

1. **needs + if**

   Sin pruebas en verde o fuera de main, no hay despliegue.

2. **environment: production**

   Puede exigir la aprobación de una persona.

3. **OIDC (id-token)**

   Credenciales temporales: cero llaves guardadas.

4. **s3 sync --delete**

   Publica el build y elimina archivos viejos.



El backend sigue el mismo patrón: imagen Docker → registro → servicio.

Smoke test post-despliegue y rollback

New Horizons
Centro de Capacitación

.github/workflows/ci-cd.yml

```yaml
      - name: Smoke test post-despliegue
        run: |
          for i in 1 2 3 4 5; do
1           curl -fs "${{ vars.API_URL }}/health" && exit 0
            echo "Intento $i fallido, reintentando..."
            sleep 10
          done
2         exit 1
```

Program.cs

```
builder.Services.AddHealthChecks();
// ...
3 app.MapHealthChecks("/health");
```

La prueba de motores

Antes de subir pasajeros, el piloto acelera en la pista. Si algo suena raro, no despega.

Despliegue terminado

GET /health → 200

Versión liberada

¿/health falla 5 veces? El job termina en rojo y el equipo se entera al instante.

Rollback = volver a ejecutar el workflow del último commit estable (Re-run jobs).

Prueba de regresión end-to-end con los 4 roles

New Horizons
Centro de Capacitación

🎭 **Ensayo general antes del estreno:** todo el elenco, con vestuario y luces reales. Recorremos el flujo completo en producción, rol por rol.

👤 Solicitante

Estado → Pendiente

- ☐ Inicia sesión
- ☐ Crea la solicitud con partidas
- ☐ Ve el total y el estado

👨‍💼 Aprobador

Estado → En revisión

- ☐ Ve su bandeja filtrada
- ☐ Aprueba o rechaza con comentario
- ☐ No ve opciones de otros roles

🏛️ Gerencia

Estado → Aprobada

- ☐ Dictamen de montos altos
- ☐ Validación cruzada del tope
- ☐ Timeline actualizado

💳 Tesorería

Estado → Pagada

- ☐ Registra el pago con voucher
- ☐ Cierra el ciclo de la solicitud
- ☐ Auditoría en logs

🔁 Cada falla encontrada en el ensayo se convierte en **una prueba automatizada nueva** del pipeline.

Checklist de salida a producción

New Horizons
Centro de Capacitación

✈️ **Checklist de pre-vuelo:** ningún piloto despega sin revisarla, aunque haya volado mil veces.

- [ ] **HTTPS activo y CORS con el dominio real**

  *Nada de localhost ni * en producción.*

- [ ] **Secretos solo en GitHub y en la nube**

  El repositorio no contiene ninguna clave.

- [ ] **Migraciones aplicadas con backup previo**

  La base de datos tiene vuelta atrás.

- [ ] **Logs y /health funcionando**

  Si algo falla, te enteras antes que el usuario.

- [ ] **Branch protection en main**

  Nadie despliega saltándose el pipeline.

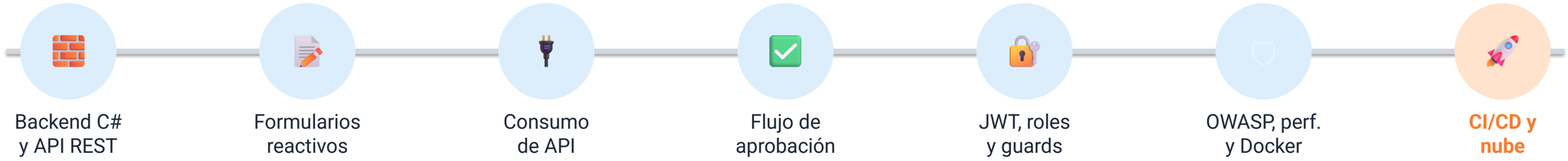
- [ ] **Rollback probado al menos una vez**

  Sabes volver atrás en minutos.

Retrospectiva final: el viaje del curso

New Horizons
Centro de Capacitación

Antes de bajar del tren, miramos el mapa del viaje: todo lo que construimos, estación por estación.



Seguir haciendo

¿Qué práctica del curso te llevas a tu trabajo desde mañana?

✍️ Escribe tu respuesta en una nota

Mejorar

¿Qué parte del proyecto reescribirías si empezaras hoy?

✍️ Escribe tu respuesta en una nota

Empezar a hacer

¿Qué tema quieres profundizar por tu cuenta?

✍️ Escribe tu respuesta en una nota

Errores comunes y cómo evitarlos

New Horizons®
Centro de Capacitación

✗ Evita esto	✓ Haz esto
Omitir la validación estricta de parámetros en el servidor	Validar siempre los contratos DTO en cada llamada HTTP
Almacenar información sensible en repositorios públicos	Usar GitHub Secrets y variables de entorno seguras
Desacoplar la UI de las políticas de seguridad del backend	Derivar los permisos de la UI de los mismos claims del backend
Desplegar a mano desde la laptop del desarrollador	Desplegar solo desde el pipeline, con trazabilidad por commit
Hacer merge a main con los checks en rojo	Activar branch protection con status checks obligatorios

Próximos pasos: tu ruta Full Stack

New Horizons
Centro de Capacitación

Nivel desbloqueado. Terminaste el curso; ahora eliges tu próximo mundo. Seis caminos para seguir creciendo.

Orquestación

Kubernetes o Amazon ECS para escalar los contenedores de la Sesión 18.

Observabilidad

OpenTelemetry, métricas y dashboards sobre los logs de Serilog.

Pruebas E2E

Playwright o Cypress para automatizar la regresión de los 4 roles.

Arquitectura

Mensajería, microservicios y patrones como CQRS cuando el negocio lo pida.

Certificación cloud

Por ejemplo, AWS Certified Developer – Associate.

Portafolio

Publica este proyecto en GitHub con README, demo desplegada y pipeline en verde.

Resumen de la Sesión 19

New Horizons®
Centro de Capacitación

🔄 GitHub Actions

Automatiza pruebas y builds en cada commit garantizando calidad continua.

≈ La línea de ensamblaje

🚀 Despliegue automatizado

Reduce errores manuales y publica nuevas versiones en minutos.

≈ El camión de reparto

👤 Regresión E2E

Confirma el éxito del proyecto con los 4 roles corporativos.

≈ El ensayo general

🎯 Ciclo completo

Diseño, backend, frontend y DevOps: dominas el ciclo de vida del software.

≈ El mapa del viaje

🔒 Listo para producción

Una solución enterprise, segura y desplegada.

≈ La checklist de pre-vuelo

🎉 ¡Felicitaciones!

Culminaste el Curso Técnico Full Stack con C# y Angular.

≈ Nivel desbloqueado

PREGUNTAS...

New Horizons®
Centro de Capacitación

1. ¿Cómo previene un pipeline de CI que código roto llegue a la rama principal de producción?
2. ¿Cómo se gestionan de forma segura los secretos de despliegue en GitHub Secrets?
3. ¿Qué diferencias existen entre desplegar en App Services administrados vs servidores VPS?
4. ¿Cómo verificamos que todas las reglas de negocio de la máquina de estados funcionen en producción?
5. ¿Cuáles son los siguientes pasos recomendados para continuar el crecimiento profesional Full Stack?

New Horizons
Centro de Capacitación

Conclusiones



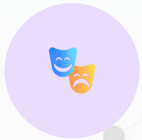
Calidad continua

GitHub Actions automatiza pruebas y builds en cada commit garantizando calidad continua.



Entregas en minutos

El despliegue automatizado reduce errores manuales y permite publicar nuevas versiones en minutos.



Ensayo superado

La prueba de regresión integral confirma el éxito del proyecto con los 4 roles corporativos.

Ciclo completo

El estudiante domina el ciclo completo de vida del software: diseño, backend, frontend y DevOps.



¡Misión cumplida!

¡Felicitaciones por culminar con éxito el Curso Técnico Full Stack con C# y Angular!

GRACIAS